

Binary Digits

 **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

This file doesn't look like much... just a bunch of 1s and 0s. But maybe it's not just random noise. Can you recover anything meaningful from this?

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable strings
 - `exiftool` – Read metadata
 - `python3` – Custom bit-to-byte conversion script
 - `cp` – Copy / rename files
-

Complete Solution

Step 1: Download and Prepare

First, download the file named `digits.bin` from the challenge page.

Step 2: Basic File Inspection

Let's check what kind of file we're dealing with:

```
file digits.bin
```

Expected output:

```
digits.bin: ASCII text, with very long lines (65536), with no line terminators
```

The file is actually **plain ASCII text** containing a long sequence of `0` and `1` characters, not a binary file as the extension suggests.

Step 3: Search for Visible Text

Before diving into decoding, let's see if the flag is directly visible as plain text:

```
strings digits.bin | grep -i "picoCTF"
```

Note: If no results appear, proceed to the next step (decoding). In this case, the flag is not directly visible.

Step 4: Inspect Metadata

Use `exiftool` to examine the file metadata for any hidden clues:

```
exiftool digits.bin
```

Complete output:

```
ExifTool Version Number      : 12.76
File Name                    : digits.bin
Directory                    : .
File Size                    : 71 kB
File Modification Date/Time   : 2026:05:09 05:44:11+01:00
File Access Date/Time        : 2026:05:09 05:44:11+01:00
File Inode Change Date/Time   : 2026:05:09 05:44:28+01:00
File Permissions              : -rwxrwxrwx
File Type                    : TXT
File Type Extension          : txt
MIME Type                    : text/plain
MIME Encoding                 : us-ascii
```

Newlines : (none)
Line Count : 1
Word Count : 1

Key findings:

- **File Type:** `TXT` – not a binary file despite the `.bin` extension
- **Line Count:** `1` – the entire file is a single line of text
- **MIME Encoding:** `us-ascii` – contains only ASCII characters (0s and 1s)

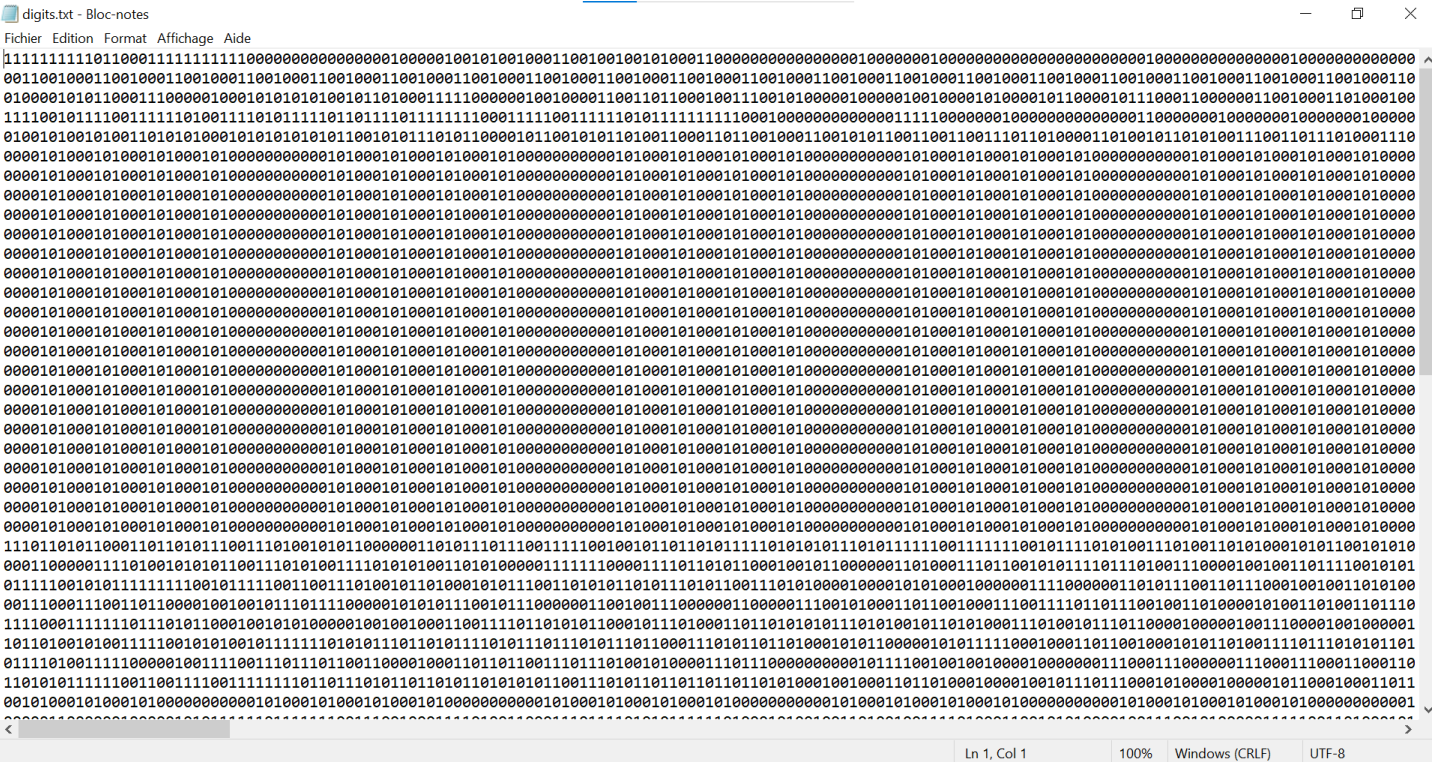
This confirms we have a long string of binary digits.

Step 5: Convert Binary to Bytes

Since the file contains a continuous stream of `0` and `1` characters, we need to group them into 8-bit chunks (bytes) and convert each chunk to its corresponding byte value.

First, make a copy with a `.txt` extension for easier handling:

```
cp digits.bin digits.txt
```



Now, let's write a Python script to perform the conversion:

```
# Read the binary string from the file
bits = open("digits.txt").read().strip()

# Group into 8-bit chunks and convert to bytes
data = bytes(int(bits[i:i+8], 2) for i in range(0, len(bits), 8))

# Save the reconstructed binary data
open("real.bin", "wb").write(data)
```

What this script does:

- Reads the entire binary string
- Splits it into chunks of 8 characters
- Converts each chunk (e.g., "01000001") to its integer value (65)
- Assembles the bytes into a new file

Save the script (e.g., as `convert.py`) and run it:

```
python3 convert.py
```

This generates a new file called `real.bin` .

Step 6: Identify the Reconstructed File

Check the type of the newly created file:

```
file real.bin
```

Expected output:

```
real.bin: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment
length 16, baseline, precision 8, 800x500, components 3
```

Observation: The reconstructed data is actually a **JPEG image** (800×500 pixels).

Step 7: View the Image

Rename the file to give it the correct JPEG extension:

```
cp real.bin img.jpg
```

Then open the image with your preferred viewer:

```
# On Linux
```

```
xdg-open img.jpg
```

```
# On macOS
```

```
open img.jpg
```

```
# On Windows
```

```
start img.jpg
```

```
picoCTF[h1dd3n_1n_th3_b1n4ry_67bd9b59]
```

The image contains the flag!

Final Result

```
picoCTF{<REDACTED>}
```

Note: The actual flag is redacted here. In the real challenge, the image will display the complete flag.

Key Concepts

- **Binary Representation:** Data is stored as sequences of 0s and 1s. Eight bits form one byte. |
- **File Signatures (Magic Bytes):** Different file formats have unique headers (e.g., JPEG starts with `FF D8`).
- **Bit-to-Byte Conversion:** Grouping binary digits into 8-bit chunks and converting them to byte values reconstructs the original file.
- **Forensics Analysis:** Inspecting file metadata and verifying file types helps identify hidden data.